East West University

CSE 420 – Computer Graphics

LAB-7 Report

Implementation of Circle Drawing Algorithms

Course Instructor: Dr. Md. Tauhid Bin Iqbal
Assistant Professor, Dept. of Computer Science & Engineering

1. Midpoint Circle Drawing Algorithm

Introduction

The Midpoint Circle Drawing Algorithm is a rasterization technique in computer graphics used to draw a circle efficiently by exploiting the circle's 8-way symmetry. Instead of computing trigonometric functions, it determines the closest pixel to the theoretical circle path by checking the midpoint between two candidate pixels. This reduces computational cost and is widely used in low-level graphics rendering.

Key Formula

Initial decision parameter: P0 = 1 - r

If Pk < 0: Pk+1 = Pk + 2x + 1 (y unchanged)

If Pk >= 0: Pk+1 = Pk + 2x - 2y + 1 (y decremented by 1)

C++ Source Code

```
#include <stdio.h>
#include <GL/gl.h>
#include <GL/glut.h>

float cx, cy, r;

void plotCirclePoints(float x, float y)
{
glVertex2f((cx + x) / 100, (cy + y) / 100);
glVertex2f((cx - x) / 100, (cy + y) / 100);
glVertex2f((cx + x) / 100, (cy - y) / 100);
glVertex2f((cx - x) / 100, (cy - y) / 100);
```

```
glVertex2f((cx + y) / 100, (cy + x) / 100);
glVertex2f((cx - y) / 100, (cy + x) / 100);
glVertex2f((cx + y) / 100, (cy - x) / 100);
glVertex2f((cx - y) / 100, (cy - x) / 100);
}

void display(void)
{
glClear(GL_COLOR_BUFFER_BIT);
glColor3f(1.0, 1.0, 1.0);
glPointSize(3.0f);
glBegin(GL_POINTS);
float x = 0;
float y = r;
int p = 1 - (int)r;
plotCirclePoints(x, y);
while (x < y)
{
x = x + 1;
if (p < 0)
p = p + 2 * (int)x + 1;
else
{
y = y - 1;
p = p + 2 * (int)x - 2 * (int)y + 1;
}
plotCirclePoints(x, y);
}
glEnd();
glFlush();
}

void printTable(void)
{
float x = 0;
float y = r;
int p = 1 - (int)r;
printf("Midpoint Circle Drawing Algorithm\n");
printf("Center: (%.0f, %.0f) Radius: %.0f\n\n", cx, cy, r);
printf("Initial point: (%.0f, %.0f) P0 = %d\n\n", x, y, p);
printf("Step x y Pk\n");
printf(" ------------------------------ \n");
int step = 0;
while (x < y)
{
int currentPk = p;
x = x + 1;
if (p < 0)
p = p + 2 * (int)x + 1;
else
{
```

```
y = y - 1;
p = p + 2 * (int)x - 2 * (int)y + 1;
}
printf("%-5d %-10.0f %-10.0f %-5d\n", step, x, y, currentPk);
step++;
}
}

void init(void)
{
glClearColor(0.0, 0.0, 0.0, 0.0);
glMatrixMode(GL_PROJECTION);
glLoadIdentity();
glOrtho(-1.5, 1.5, -1.5, 1.5, -1.0, 1.0);
}

int main(int argc, char** argv)
{
printf("Enter center coordinates (cx cy): ");
scanf("%f %f", &cx, &cy);
printf("Enter radius r: ");
scanf("%f", &r);
printf("\n");
printTable();
glutInit(&argc, argv);
glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
glutInitWindowSize(600, 600);
glutInitWindowPosition(100, 100);
glutCreateWindow("Midpoint Circle Drawing Algorithm");
init();
glutDisplayFunc(display);
glutMainLoop();
return 0;
}
```

Output

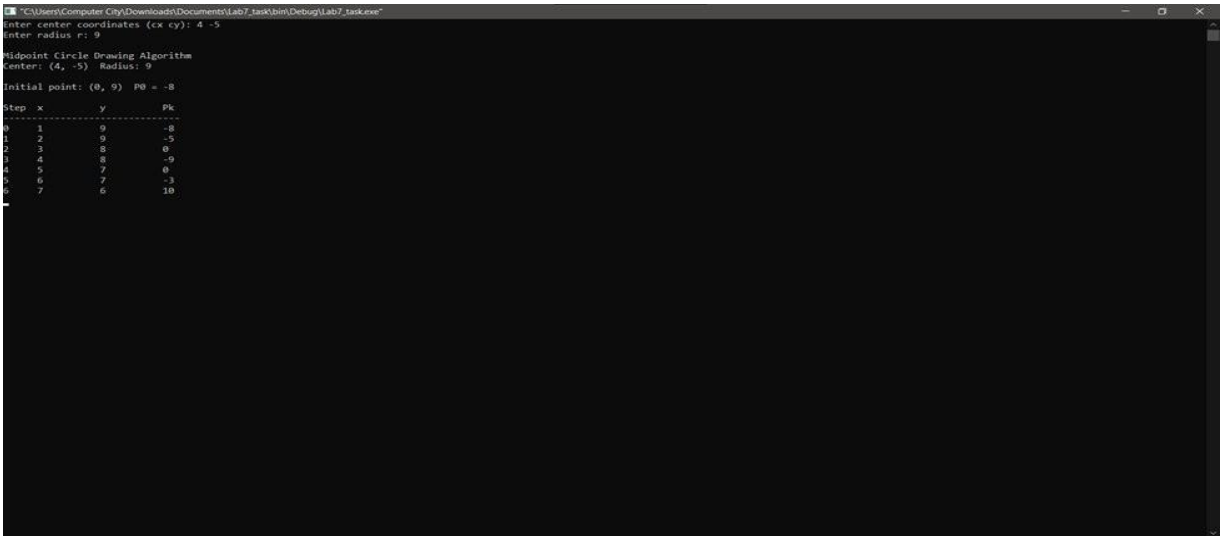

Figure 1.1 – Console output showing step-by-step table (Center: 4, -5 | Radius: 9)

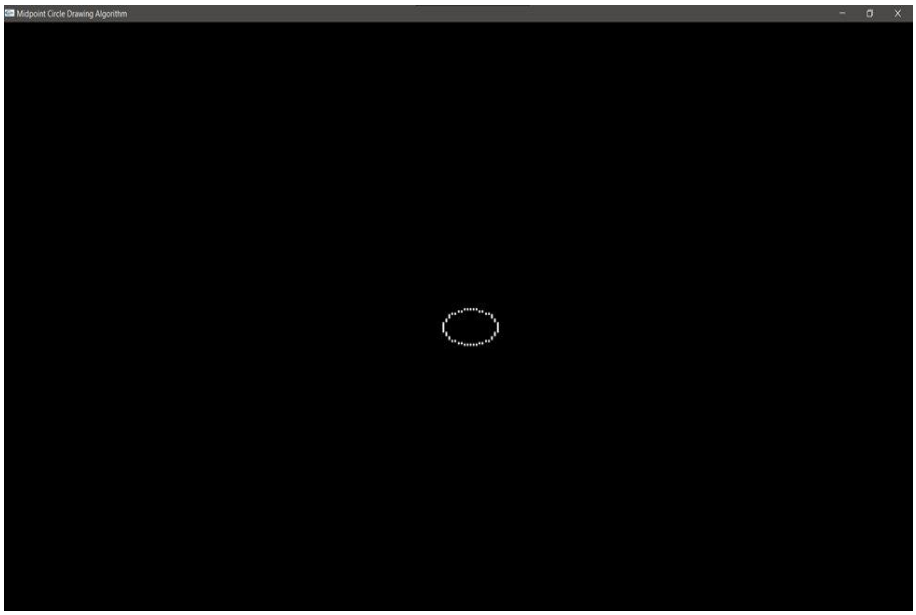

Figure 1.2 – OpenGL window rendering the circle

2. Bresenham's Circle Drawing Algorithm

Introduction

Bresenham's Circle Drawing Algorithm is an integer-only version of the midpoint circle algorithm, optimized for digital displays. It replaces the floating-point decision parameter with pure integer calculations, making it faster and more suitable for low-level hardware. It was developed by Jack Bresenham as an extension of his famous line drawing algorithm.

Key Formula

Initial decision parameter: $P0 = 3 - 2r$

If $Pk < 0$: $Pk+1 = Pk + 4x + 6$ (y unchanged)

If Pk >= 0: Pk+1 = Pk + 4(x - y) + 10 (y decremented by 1)

C++ Source Code

```
#include <stdio.h>
#include <GL/gl.h>
#include <GL/glut.h>

float cx, cy, r;

void plotCirclePoints(float x, float y)
{
glVertex2f((cx + x) / 100, (cy + y) / 100);
glVertex2f((cx - x) / 100, (cy + y) / 100);
glVertex2f((cx + x) / 100, (cy - y) / 100);
glVertex2f((cx - x) / 100, (cy - y) / 100);
glVertex2f((cx + y) / 100, (cy + x) / 100);
glVertex2f((cx - y) / 100, (cy + x) / 100);
glVertex2f((cx + y) / 100, (cy - x) / 100);
glVertex2f((cx - y) / 100, (cy - x) / 100);
}

void display(void)
{
glClear(GL_COLOR_BUFFER_BIT);
glColor3f(1.0, 1.0, 1.0);
glPointSize(3.0f);
glBegin(GL_POINTS);
float x = 0;
float y = r;
int Pk = 3 - 2 * (int)r;
plotCirclePoints(x, y);
while (x < y)
{
x = x + 1;
if (Pk < 0)
Pk = Pk + 4 * (int)x + 6;
else
{
y = y - 1;
Pk = Pk + 4 * ((int)x - (int)y) + 10;
}
plotCirclePoints(x, y);
}
glEnd();
glFlush();
}

void printTable(void)
{
float x = 0;
```

```
float y = r;
int Pk = 3 - 2 * (int)r;
printf("Bresenham Circle Drawing Algorithm\n");
printf("Center: (%.0f, %.0f) Radius: %.0f\n\n", cx, cy, r);
printf("Initial point: (%.0f, %.0f) P0 = %d\n\n", x, y, Pk);
printf("Step x y Pk\n");
printf(" ------------------------------ \n");
int step = 0;
while (x < y)
{
int currentPk = Pk;
x = x + 1;
if (Pk < 0)
Pk = Pk + 4 * (int)x + 6;
else
{
y = y - 1;
Pk = Pk + 4 * ((int)x - (int)y) + 10;
}
printf("%-5d %-10.0f %-10.0f %-5d\n", step, x, y, currentPk);
step++;
}
}

void init(void)
{
glClearColor(0.0, 0.0, 0.0, 0.0);
glMatrixMode(GL_PROJECTION);
glLoadIdentity();
glOrtho(-1.5, 1.5, -1.5, 1.5, -1.0, 1.0);
}

int main(int argc, char** argv)
{
printf("Enter center coordinates (cx cy): ");
scanf("%f %f", &cx, &cy);
printf("Enter radius r: ");
scanf("%f", &r);
printf("\n");
printTable();
glutInit(&argc, argv);
glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
glutInitWindowSize(600, 600);
glutInitWindowPosition(100, 100);
glutCreateWindow("Bresenham Circle Drawing Algorithm");
init();
glutDisplayFunc(display);
glutMainLoop();
return 0;
}
```

Output

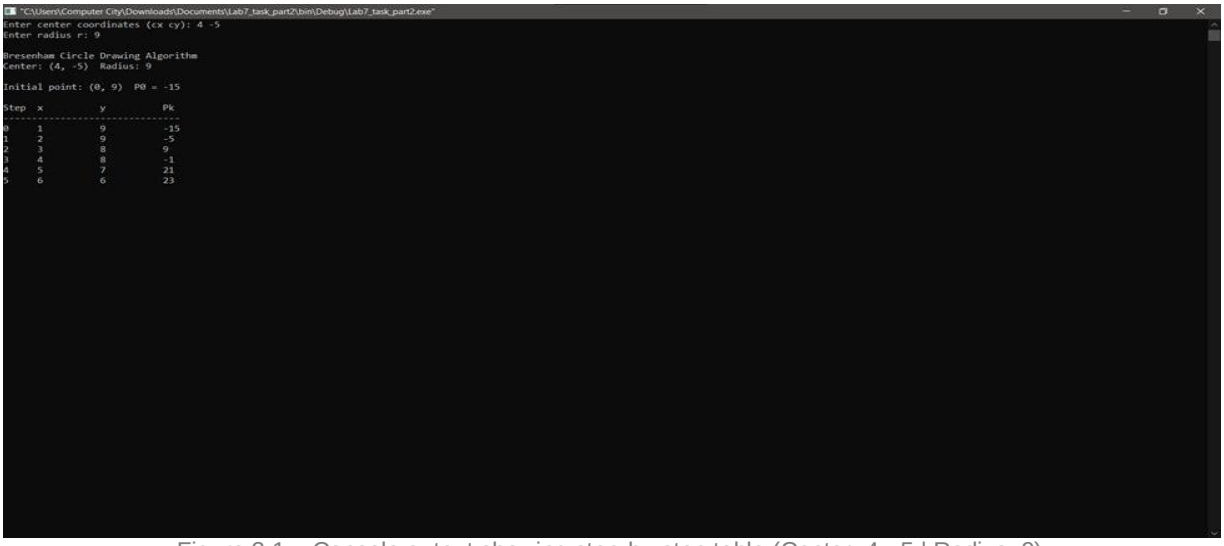

Figure 2.1 – Console output showing step-by-step table (Center: 4, -5 | Radius: 9)

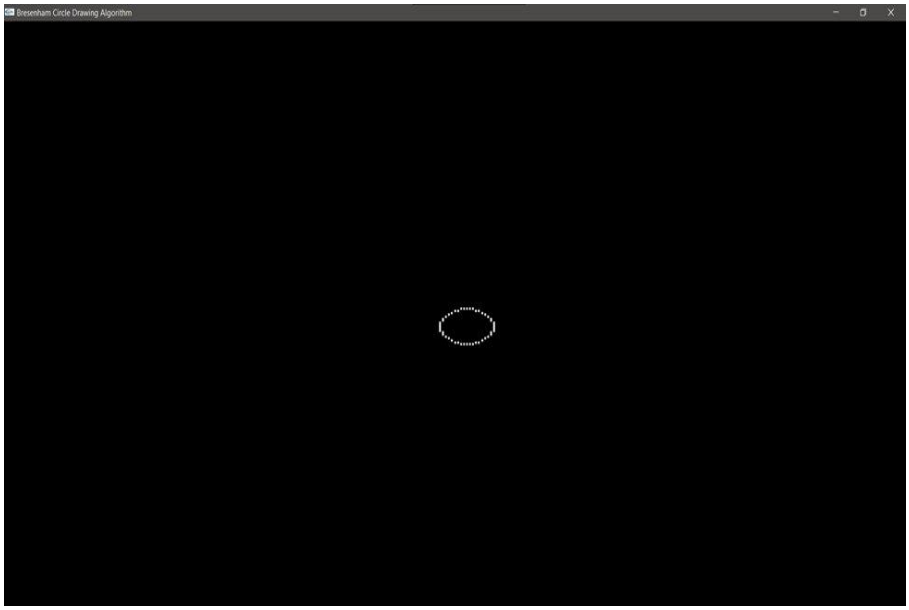

Figure 2.2 – OpenGL window rendering the circle

3. Comparison of Both Algorithms

Criteria	Midpoint Circle Algorithm	Bresenham's Circle Algorithm
Initial Decision Parameter	P0 = 1 - r	P0 = 3 - 2r
Update Formula (Pk < 0)	Pk + 2x + 1	Pk + 4x + 6
Update Formula (Pk >= 0)	Pk + 2x - 2y + 1	Pk + 4(x-y) + 10

Criteria	Midpoint Circle Algorithm	Bresenham's Circle Algorithm
Arithmetic Type	Integer (small multipliers)	Integer (larger multipliers)
Number of Steps (r = 9)	7 steps	6 steps
Pk Range (r = 9)	-8 to +10	-15 to +23
Symmetry Used	8-way symmetry	8-way symmetry
Starting Point	(0, r)	(0, r)
Loop Condition	x < y	x < y
Floating Point Needed	No	No
Visual Output	Correct circle shape	Correct circle shape
Pixel Accuracy	High	High
Speed	Fast	Slightly faster on some hardware due to simpler constants
Origin	Based on midpoint decision test	Extended from Bresenham's line algorithm
Best Use Case	General purpose circle drawing	Low-level hardware without FPU